

Министерство науки и высшего образования Российской Федерации

Самарский национальный исследовательский университет
имени академика С.П. Королева

Рекуррентные нейронные сети

Методические указания к лабораторной работе № 3
по курсу «НЕЙРОННЫЕ СЕТИ И ГЛУБОКОЕ ОБУЧЕНИЕ»

САМАРА

2026

Составители: д.ф.-м.н., профессор В.В. Мясников
к.т.н. А.Ю. Баврина

УДК 681.3

Рекуррентные нейронные сети

Методические указания к лабораторной работе № 3
Самарский национальный исследовательский университет
имени академика С.П. Королева
Составители: В.В. Мясников, А.Ю. Баврина
Самара, 2026. 16 с.

В лабораторной работе № 3 «Рекуррентные нейронные сети» дисциплины «Нейронные сети и глубокое обучение» изучаются принципы работы и основные функции построения, обучения, оптимизации и оценки качества работы рекуррентных нейронных сетей.

Методические указания предназначены для студентов, обучающихся по специализациям 10.05.03 - "Информационная безопасность автоматизированных систем" и 01.03.02 - "Прикладная математика и информатика".

Печатается по решению редакционно-издательского совета
Самарского национального исследовательского университета
имени академика С.П. Королева

Рецензент: д.ф.-м.н., профессор ФИО

Цель работы: изучить принципы работы и основные функции построения, обучения, оптимизации и оценки качества работы рекуррентных нейронных сетей.

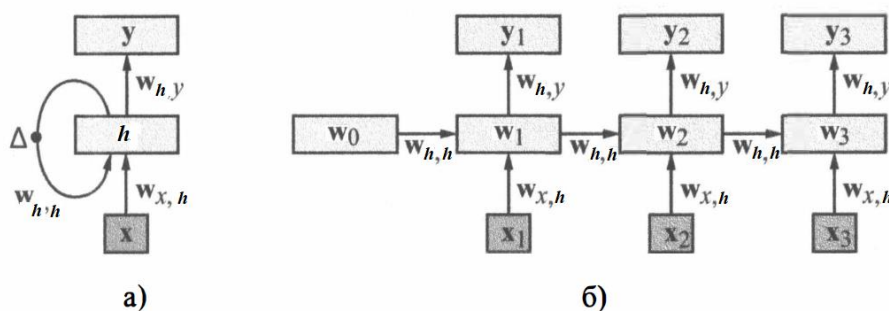
Рекуррентные нейронные сети (РНС, англ. RNN) - модели нейронных сетей с обратной связью между различными слоями нейронов. РНС отображает пространство входных последовательностей в пространство выходных последовательностей, сохраняя информацию о состоянии. Поэтому предсказание выхода зависит как от входа, так и от скрытого состояния системы, которое обновляется со временем по мере обработки последовательности. РНС можно использовать для генерирования, классификации и преобразования последовательностей.

1. Простейшие РНС и ГНС

1.1 Структуры простейшей РНС

Простейшая РНС “Vanilla RNN” изображена на рисунке 1а. Здесь:

- входной вектор x ,
- вектор скрытого состояния h ,
- выходной вектор y .



а) Принципиальная схема базовой рекуррентной нейронной сети, в которой скрытый слой z включает рекуррентные соединения, — символ Δ здесь указывает на задержку. б) Та же самая сеть, развернутая по трем временным этапам для создания сети с прямой связью. Обратите внимание, что одни и те же веса используются на всех временных этапах

Рисунок 1 - Простейшая РНС “Vanilla RNN”

Поскольку подача входных данных производится последовательно, все указанные выше векторы меняются со временем, поэтому удобно использовать обозначение с постфиксом (t) для обозначения момента дискретного времени $t=0,1,\dots,T-1$

Получив текущей по времени входной вектор $x(t)$ сеть пересчитывает вектор состояния и выходной вектор по следующим формулам:

$$\begin{aligned} h(t) &= f_h(W_{hh}h(t-1) + W_{xh}x(t) + b_h), \\ y(t) &= f_y(W_{hy}h(t) + b_y). \end{aligned} \quad (1)$$

Здесь матрицы W_{xh} , W_{hh} , W_{hy} и b_h , b_y - обучаемые/настраиваемые параметры РНС (матрицы и векторы смещений), связывающие, соответственно, входной вектор с вектором состояния, два последовательных (по времени) скрытых состояния, а

также скрытое состояние с выходным вектором.

Таким образом, получив из входной последовательности входной вектор $x(t)$ сеть выдала в выходную последовательность вектор $y(t)$ и сменила свое скрытое состояние с $h(t-1)$ на $h(t)$. Теоретически нет предела тому, насколько далеко назад во входных данных может заглядывать модель, поэтому считается, что архитектура РНС позволяет решить проблему ограниченного контекста.

Выбор активационных функций для выходного вектора РНС зависит от того, с чем именно работает РНС. Если выходная последовательность – слова или токены, то в качестве выхода удобно использовать softmax с размерностью, совпадающей с размерностью словаря. Если выход РНС вещественный – то можно выбирать типовые функции типа ReLU, сигмоиды и др.

Режим использования РНС также зависит от решаемой задачи. Основное множество режимов использования представлено на рисунке 2 ниже.

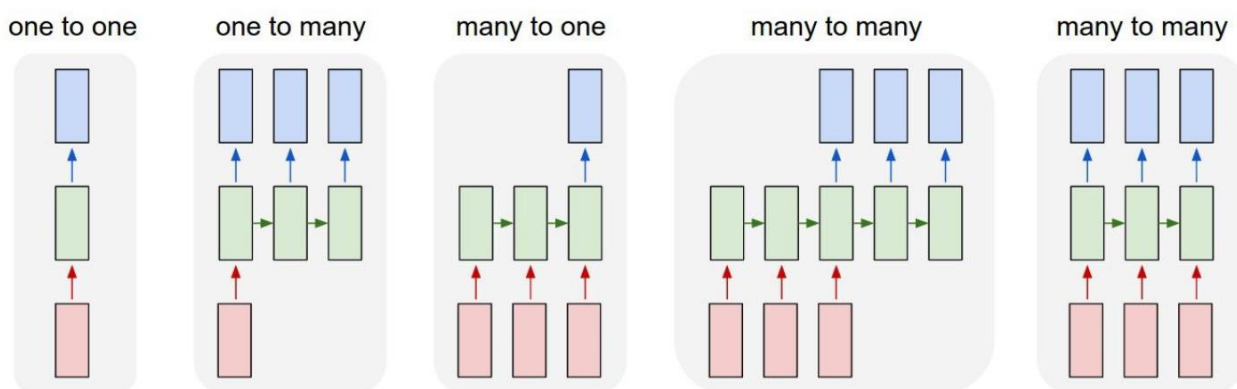


Рисунок 2 – Режимы работы РНС

1.2 Глубокие РНС

Стандартный метод построения глубоких РНС поразительно прост - мы накладываем РНС друг на друга. По входным данным первый РНС выдает последовательность выходных данных определенной длины. Они, в свою очередь, представляют собой входные данные для следующего уровня РНС. На рисунке ниже дана иллюстрация глубокой РНС с несколькими скрытыми слоями. Каждое скрытое состояние оперирует последовательным вводом и выдает последовательный вывод. Более того, любая ячейка РНС (белый прямоугольник на рис) на каждом временном шаге зависит как от значения того же слоя на предыдущем временном шаге, так и от значения предыдущего слоя на том же временном шаге.

Формально, используя аналогичные выражению (1) обозначения (добавим явно смещения):

$$\begin{aligned} h^k(t) &= f_h^k(W_{hh}^k h^k(t-1) + W_{xh}^k h^{k-1}(t) + b_h^k), \\ y(t) &= f_y(W_{hy} h^{K-1}(t) + b_y). \end{aligned} \quad (2)$$

Здесь K – число слоев, индекс $k = \overline{0, K-1}$. Матрицы W_{hh}^k связывают состояния по времени на одном слое, как в соотношениях (2), а матрицы W_{xh}^k , наоборот, связывают одномоментные состояния по глубине РНС. При этом полагаем $h^{-1}(t) = x(t)$, то есть сеть имеет ровно K скрытых состояний. В качестве функции f_y для задач

классификации обычно используется функция softmax, а для задач регрессии – тождественная.

Следует также отметить, что при обработке пакетов/батчей, вычисления (2) можно выполнять и в пакетном режиме. Для этого векторы входов, состояний и выходов преобразуются в матрицы.

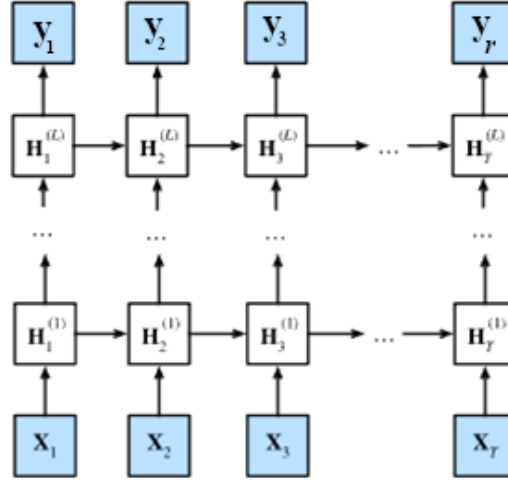


Рисунок 3 - Архитектура глубокой РНС

1.3 Обучение РНС, процедура обратного распространения

Выпишем процедуру обратного распространения для простейшей РНС, выражения для которой приведены в (1). Сделаем дополнительные обозначения:

$$\begin{aligned}\tilde{h}(t) &= W_{hh}h(t-1) + W_{xh}x(t) + b_h, \quad h(t) = f_h(\tilde{h}(t)), \\ \tilde{y}(t) &= W_{hy}h(t) + b_y, \quad y(t) = f_y(\tilde{y}(t)).\end{aligned}$$

Пусть критерий имеет следующий вид:

$$L = \frac{1}{T} \sum_{t=0}^{T-1} l(y^{true}(t), y(t)). \quad (3)$$

Частная производная $\frac{\partial L}{\partial W_{hy}}$

$$\frac{\partial L}{\partial W_{hy}} = \frac{1}{T} \sum_{t=0}^{T-1} l'(y^{true}(t), y(t)) f'_y(\tilde{y}(t)) h(t).$$

Частная производная $\frac{\partial L}{\partial b_y}$

$$\frac{\partial L}{\partial b_y} = \frac{1}{T} \sum_{t=0}^{T-1} l'(y^{true}(t), y(t)) f'_y(\tilde{y}(t)).$$

Частная производная $\frac{\partial L}{\partial W_{hh}}$

$$\frac{\partial L}{\partial W_{hh}} = \frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial (l(y^{true}(t), y(t)))}{\partial W_{hh}} = \frac{1}{T} \sum_{t=0}^{T-1} l'(y^{true}(t), y(t)) f'_y(\tilde{y}(t)) W_{hy} \frac{\partial (h(t))}{\partial W_{hh}}$$

В свою очередь, частная производная состояния может быть получена следующим образом, учитывая рекуррентную зависимость от предшествующего состояния:

$$\frac{\partial h(t)}{\partial W_{hh}} = f'_h(\tilde{h}(t)) \left(h(t-1) + W_{hh} \frac{\partial h(t-1)}{\partial W_{hh}} \right) \quad (3)$$

Частная производная $\frac{\partial L}{\partial W_{xh}}$

$$\frac{\partial L}{\partial W_{xh}} = \frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial (l(y^{true}(t), y(t)))}{\partial W_{xh}} = \frac{1}{T} \sum_{t=0}^{T-1} l'(y^{true}(t), y(t)) f'_y(\tilde{y}(t)) W_{hy} \frac{\partial (h(t))}{\partial W_{xh}}$$

где

$$\frac{\partial h(t)}{\partial W_{xh}} = f'_h(\tilde{h}(t)) \left(W_{hh} \frac{\partial h(t-1)}{\partial W_{xh}} + x(t) \right).$$

Частная производная $\frac{\partial L}{\partial b_h}$

$$\frac{\partial L}{\partial b_h} = \frac{1}{T} \sum_{t=0}^{T-1} \frac{\partial (l(y^{true}(t), y(t)))}{\partial b_h} = \frac{1}{T} \sum_{t=0}^{T-1} l'(y^{true}(t), y(t)) f'_y(\tilde{y}(t)) W_{hy} \frac{\partial (h(t))}{\partial b_h}$$

где

$$\frac{\partial h(t)}{\partial b_h} = f'_h(\tilde{h}(t)) \left(W_{hh} \frac{\partial h(t-1)}{\partial b_h} + 1 \right).$$

Участвующие во всех представленных выражениях производные показателя рассогласования ответов $l'(\dots, \dots)$ и производные активационных функций f'_* рассматривались в лекциях по обучению ГНС и лекциях по сверточным сетям. Напомним, что для случая критерия в виде кросс-энтропии и активационной функции softmax на выходе, производная критерия по аргументу активационной функции имеет вид $(y_k(t) - p_k^{true})$ — k -ый элемент вектора $y(t)$, p_k^{true} — элементы вектора истинной классификации):

$$l'(y^{true}(t), y(t)) f'_y(\tilde{y}(t)) = \begin{cases} y_k(t) - 1, & p_k^{true} = 1; \\ y_k(t), & p_k^{true} \neq 1. \end{cases}$$

Очевидно, что выражения для градиентов являются рекурсивными: вклад в градиент от временного этапа t рассчитывается с использованием вклада от временного этапа $t-1$. При этом общее время выполнения расчетов по вычислению градиента будет линейно пропорционально числу тактов сети. Такой алгоритм — *обратное распространение ошибки во времени* (англ.: *back-propagation through time* — BPTT).

Можно также заметить, что градиенты в конечный момент времени T будут включать термы (элементы выражений), пропорциональные $(f'_h(\cdot) W_{hh})^T$. Для типовых активационных функций типа сигмиды, tanh или ReLU величина $f'_h(\cdot) \leq 1$, поэтому простая сеть РНС будет страдать от проблемы исчезновения градиента, если *спектральная норма*¹ матрицы W_{hh} будет меньше единицы. Обратно, если спектральная норма матрицы W_{hh} окажется больше единицы, то возникает проблема *взрывного градиента*. Таким образом, наибольшее по модулю собственное значение матрицы весов W_{hh} существенно определяет поведение градиентов и, следовательно, получаемый результат обучения.

¹ Спектральная норма оператора равна максимальному сингулярному числу этого оператора. Спектральная норма нормального оператора равна абсолютному значению максимального по модулю собственного значения этого оператора. Спектральный радиус — для квадратной матрицы определяется как максимум абсолютных значений её собственных значений.

Для решения проблемы исчезающих или взрывообразных градиентов обычно используется простой эвристический прием - значение градиентов усекается по некоторому максимальному значению (норма градиента ограничивается некоторой величиной). Также, как более сложное решение, может контролироваться спектральный радиус λ прямого отображения, W_{hh} , а также обратного отображения, описываемого якобианом J_{hh} .

*Альтернатива явному контролю над спектральной нормой (=сингулярным числом) матрицы W_{hh} является модификация самой архитектуры РНС, а именно добавление **аддитивных**, а не мультипликативных обновлений скрытых состояний (см. LSTM).*

1.4 Преимущества и недостатки простейших РНС

Преимущества RNN:

1. Можно обрабатывать входные данные любой длины;
2. При вычислении для шага t можно (теоретически) использовать информацию, полученную за много шагов назад;
3. Размер модели не увеличивается при более длинном вводе;
4. Одинаковые веса применяются на каждом временном шаге, поэтому существует симметрия в том, как обрабатываются входные данные.

Недостатки RNN:

1. Рекуррентные вычисления выполняются медленно;
2. На практике трудно получить доступ к информации, сделанной на много шагов назад – то есть невозможность долго держать всю информацию о последовательности/предложении (большее влияние на результат всегда оказывают последние входные данные).
3. Проблемы исчезающих и взрывающихся градиентов.
4. В качестве дополнительного недостатка, неочевидного на данный момент, можно указать на отсутствие механизма «внимания» (англ.: *attention*).

2 АРХИТЕКТУРНЫЕ ИЗМЕНЕНИЯ ДЛЯ УЛУЧШЕНИЯ РНС

2.1 Управляемые рекуррентные блоки GRU

РНС с достаточным числом скрытых блоков в принципе могут запомнить входные данные из далекого прошлого. Но на практике так не поступают из-за проблемы исчезающего градиента. Способ решить эту проблему - обновлять скрытое состояние аддитивным способом, как в остаточной сети.

Такая идея реализована в *управляемых рекуррентных блоках* (англ.: Gated Recurrent Units - GRU) [9]. Основная идея, лежащая в основе архитектуры блока, – понять, когда следует обновлять скрытое состояние (используя управляемый блок). Это можно использовать для избирательного «запоминания» важных частей информации при первой встрече. Модель также можно обучить тому, когда сбрасывать скрытое состояние и, следовательно, забывать вещи, когда они больше не нужны.

Введем вектора для вентилей, то есть структур, которые будут использоваться для управления потоком передаваемой информации, следующим образом (это вектора, как и вектора смещений b_r):

- вентиль сброса:

$$r(t) = \sigma(W_{xr}x(t) + W_{hr}h(t-1) + b_r);$$

- вентиль обновления:

$$z(t) = \sigma(W_{xz}x(t) + W_{hz}h(t-1) + b_z).$$

В представленных выражениях функция $\sigma(\dots)$ – поэлементная сигмоидальная вещественная функция со значениями в диапазоне (0,1):

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad (1 - \sigma(x) = \sigma(-x)).$$

Далее, определим кандидата на роль вектора следующего состояния в виде ($\tanh$ также берется от каждого элемента вектора):

$$\tilde{h}(t) = \tanh(W_{xh}x(t) + W_{hh}(r(t) \circ h(t-1)) + b_h),$$

Здесь « $\circ$ » операция поэлементного умножения векторов (умножение Адамара, что означает равенство их размеров). Заметим также, что матричное произведение векторов на матрицы во всех указанных выражениях есть не что иное, как полносвязный слой сети.

Последняя формула объединяет еще не сброшенную память о прошлом (член $r(t) \circ h(t-1)$) с новым входным вектором – слагаемым $W_{xh}x(t)$. Результирующую линейную комбинацию мы передаем в функции $\tanh$, чтобы скрытые блоки оставались в интервале (-1, 1). Если элементы вентиля сброса $r(t)$ близки к «1», то мы возвращаемся к стандартному правилу обновления РНС (2). Если же они близки к «0», то модель превращается в обычный перцептрон, примененный к входному вектору. Таким образом, вентиль сброса способен контролировать краткосрочную информацию.

Выход ячейки GRU, архитектура которой приведена на рисунке 4, как и окончательное состояние, определяется на основании вектора вентиля обновления следующим образом:

$$h(t) = z(t) \circ h(t-1) + (1 - z(t)) \circ \tilde{h}(t),$$

Очевидно, при $z(t) = 1$ мы передаем предшествующее состояния без изменений и игнорируем вновь пришедший входной вектор $x(t)$. Именно так вентиль реализует долгосрочную память и долгосрочные зависимости.

Операции сложения и умножения на рисунке 4 – поэлементные. Вектора и матрицы конкатенируются по вертикали.

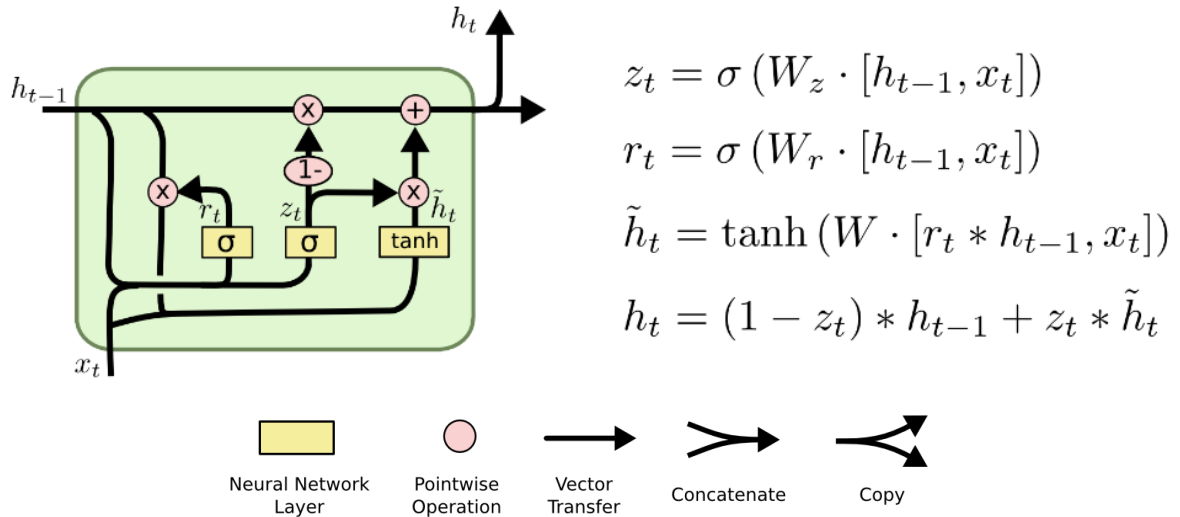


Рисунок 4 – Архитектура управляемого рекуррентного блока

2.2 Сеть долгой краткосрочной памяти (LSTM)

Модель *долгой краткосрочной памяти* (англ.: Long Short-Term Memory - LSTM) из работы [6] – более сложный вариант GRU (который, к тому же, появился почти на 20 лет раньше). Основная идея состоит в том, чтобы дополнить скрытое состояние $h(t)$ ячейкой памяти $c(t)$. Для управления этой ячейкой нам нужно три вентилей:

- выходной вентиль $o(t)$ определяет, что читать из ячейки (o - output)

$$o(t) = \sigma(W_{xo}x(t) + W_{ho}h(t-1) + b_o);$$

- входной вентиль $i(t)$ определяет, что помещать в ячейку (i - input)

$$i(t) = \sigma(W_{xi}x(t) + W_{hi}h(t-1) + b_i);$$

- вентиль забывания $f(t)$ определяет, когда сбрасывать ячейку (f - forget).

$$f(t) = \sigma(W_{xf}x(t) + W_{hf}h(t-1) + b_f);$$

Затем мы вычисляем кандидата на новое состояние ячейки:

$$\tilde{c}(t) = \tanh(W_{xc}x(t) + W_{hc}h(t-1) + b_c),$$

Новое состояние ячейки либо совпадает с вычисленным вектором-кандидатом (если входной вентиль открыт), либо совпадает со старым вектором (если вентиль забывания закрыт), либо есть их взвешенное среднее:

$$c(t) = f(t) \circ c(t-1) + i(t) \circ \tilde{c}(t).$$

Если $f(t) = 1$ и $i(t) = 0$, то ячейка реализует долгосрочную память.

Наконец, мы можем вычислить скрытое состояние как результате преобразования ячейки при условии, что выходной вентиль открыт:

$$h(t) = o(t) \circ \tanh(c(t)),$$

Заметим, что $h(t)$ является одновременно выходом блока и скрытым состоянием на следующем временном шаге. Это позволяет модели запоминать, что она только что вывела (краткосрочная память), тогда как вектор $c(t)$ работает как долгосрочная память для ячейки. Отсюда и название LSTM.

Результат работы LSTM, то есть последовательность состояний $h(t)$, используется для получения требуемого выхода $y(t)$. Обычно для этого вектор состояния преобразуется линейным образом (для регрессионных задач) и, возможно, дополнительно поступает в функцию f_y (для задач классификации выбирается softmax). Формально это можно записать:

$$y(t) = W_{hy}h(t) + b_y;$$

$$p(t) = f_y(y(t)).$$

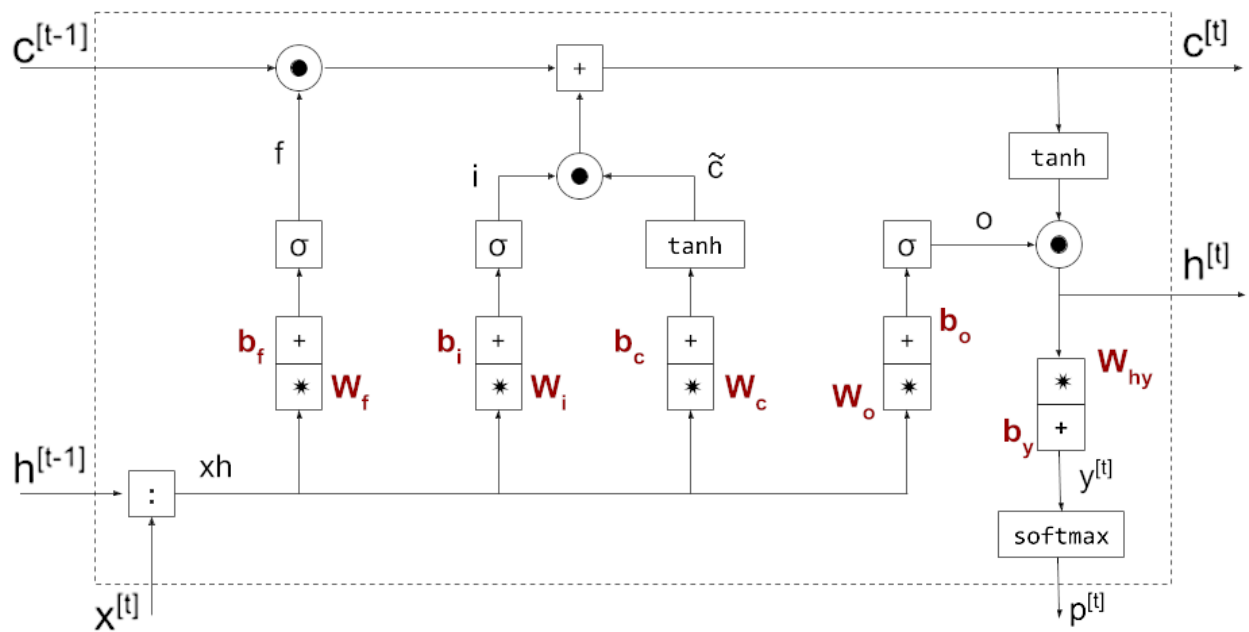


Рисунок 5 – Архитектура управляемого рекуррентного блока

Процедура обратного распространения для LSTM изложена в оригинальной работе [6] и в значительной степени повторяет ее изложение для РНС, добавляя код для нескольких вентилей вместо одного непосредственного выражения для РНС. На рисунке 6 представлена иллюстрация прохождения градиентов для LSTM.

На основании представленного описания можно сделать следующие выводы. Архитектура LSTM упрощает для РНС сохранение информации на протяжении многих временных интервалов: например, если $f(.) = 1$ и $i(.) = 0$, то информация в этой ячейке сохраняется неопределенно долго. В отличие от этого, для Vanilla-RNN сложнее получить/настроить матрицу W_{hh} , которая полностью сохраняет информацию в скрытом состоянии.

Однако, как также можно заметить и о чем указывали авторы многих работ, *LSTM не гарантирует отсутствие исчезающих/взрывающихся градиентов*, но предоставляет модели более простой способ передачи информации во времени. Также некоторые авторы указывают об упрощении прохождения градиента и поведении сети, аналогично ResNet с ее skip-соединениями (см. рисунок 6).

Uninterrupted gradient flow!

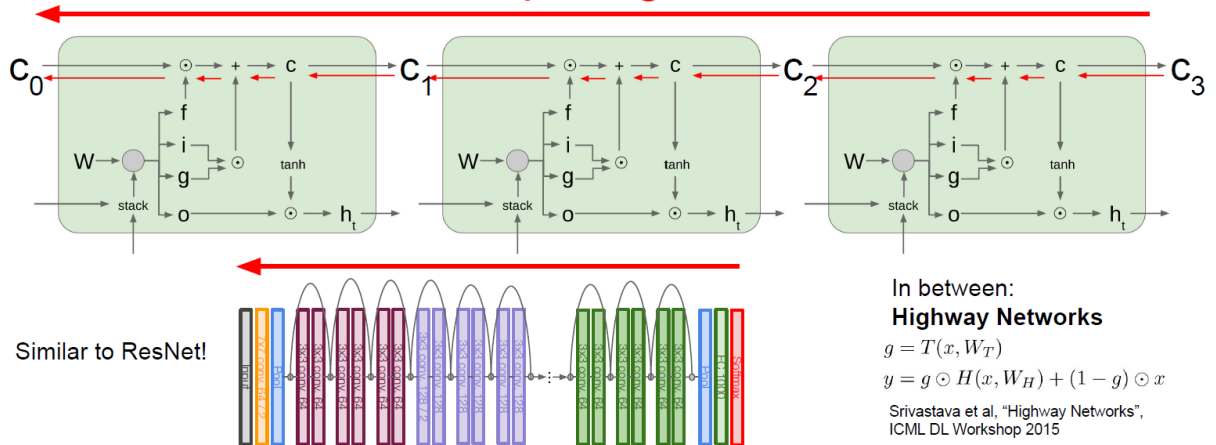


Рисунок 6 – Процесс прохождения градиентов через блоки LSTM

2.3 Литература

1. Бишоп К. М., Бишоп Х. Глубокое обучение: принципы и концепции, ДМК Пресс, 2025.
2. Принс Саймон Машинное обучение. От основ до продвинутых моделей (серия - Библиотека MIT), 2025.
3. Гудфеллоу Я., Бенджио И., А.Курвиль Глубокое обучение, ДМК Пресс, 2017
4. Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
5. Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013
6. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
7. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
8. Cho, K., Van Merriënboer, B., Bahdanau, D., & Bengio, Y. (2014). On the properties of neural machine translation: Encoder–decoder approaches. ArXiv:1409.1259.
9. Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. ArXiv:1412.3555.

3. ПОРЯДОК ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

3.1. Исходные данные

- наборы данных для числовых последовательностей `airline-passengers.csv`, `shampoo.csv`;
- наборы данных для текстовых последовательностей `input.txt`, `cnus.txt`, `cnus-clean.txt`;
- код программы, файлы: `lstm_torch.py`, `min-char-rnn.py`, `min-char-lstm.py` и данные/файлы о необходимом окружении (`env_help`).

3.2. Общий план выполнения работы²

1. Изучить код программы `lstm_torch.py` для работы с рекуррентной сетью LSTM, встроенной в фреймворк/библиотеку `torch.nn`. Запустить и проанализировать результаты для числовых последовательностей «`airline-passengers.csv`» и «`shampoo.csv`».

2. Изучить код программ `min-char-rnn.py` и `min-char-lstm.py`. Разобраться в соответствии элементов обычной РНС и сети LSTM, представленных в настоящих методических указаниях, и элементов программы. Разобраться в алгоритмах функционирования и обучения. Добавить в обе программы оценку общего времени их выполнения (обучения) и сохранение данных показателя качества на этапах обучения.

Примечание: вспомогательные комментарии по коду доступны на сайтах:

<https://eli.thegreenplace.net/2018/minimal-character-based-lstm-implementation/>
<https://mkffl.github.io/2019/07/08/minimalist-RNN.html>

3. Обучить типовую модель РНС `min-char-rnn.py` на обучающей выборке `input.txt` и `cnus-clean.txt`. Вывести (и сохранить в PDF) данные обучения для отчета (значения критерия качества и примеры работы модели), сохранить график качества обучения от номера этапа в файл.

4. Выполнить п.3 для модели `min-char-lstm.py`.

5. Считать оба графика из п.3 и п.4, показать на экране с данными времени обучения. Сохранить в файл PDF для отчета.

6. Сделать выводы по результатам исследования.

7. Сдать результаты работы и отчет и ответить на вопросы.

3.3. Содержание отчета

Отчет по работе должен содержать:

1. Исполняемые файл(ы) со всеми модификациями, допускающий запуск и выполняющий первичные шаги обучения для различных вариантов.
2. Результаты обучения и исследования – несколько файлов PDF в соответствии с приведенным выше заданием.
3. Сводный файл PDF с результатами исследования по п.5 задания.

² Рекомендуемая среда исполнения Python 3.10 с последовательно установленными модулями: `pytorch` (`torch`), `pandas`, `scikit-learn` (`sklearn`), `matplotlib`, `tqdm`, `pyarrow`. Если обучение на ноутбуках занимает чрезвычайно много времени, можно сохранять логи по эпохам для демонстрации во время показа работы.

4. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое рекуррентные нейронные сети (РНС)?
2. В чем заключается основная идея использования рекуррентных связей в нейронных сетях?
3. Какие задачи можно решать с помощью рекуррентных нейронных сетей?
4. Какова структура базового рекуррентного блока в РНС?
5. Как происходит передача информации внутри рекуррентной нейронной сети на каждом временном шаге?
6. Как обучаются рекуррентные нейронные сети?
7. Как преодолеть проблему затухающих и взрывающихся градиентов в рекуррентных сетях?
8. Как выбрать функцию активации для рекуррентных нейронных сетей?
9. Как происходит обучение рекуррентных нейронных сетей на последовательных данных?
10. Что такое LSTM (Long Short-Term Memory) в контексте РНС?
11. Какие проблемы решает механизм LSTM в сравнении с обычными рекуррентными нейронными сетями?
12. Какова структура основного LSTM блока?
13. Как LSTM сохраняет и обрабатывает долгосрочные зависимости в данных?
14. Как происходит передача информации через ворота (gates) в LSTM?
15. Какие компоненты включают в себя ворота LSTM: вход, забывания и вывода?
16. Как реализовать однонаправленные РНС в PyTorch?
16. Как реализовать однонаправленные LSTM в PyTorch?

СОДЕРЖАНИЕ

1. Простейшие РНС и ГНС	3
1.1 Структуры простейшей РНС	3
1.2 Глубокие РНС	4
1.3 Обучение РНС, процедура обратного распространения	5
1.4 Преимущества и недостатки простейших РНС	7
2 Архитектурные изменения для улучшения РНС	8
2.1 Управляемые рекуррентные блоки GRU	8
2.2 Сеть долгой краткосрочной памяти (LSTM)	9
2.3 Литература.....	11
3. Порядок Выполнения лабораторной работы.....	12
3.1. Исходные данные	12
3.2. Общий план выполнения работы.....	12
3.3. Содержание отчета.....	12
4. Контрольные вопросы.....	13
Содержание	14

Учебное издание

Рекуррентные нейронные сети
Методические указания к лабораторной работе № 3
по курсу «Нейронные сети и глубокое обучение»

Составители: Мясников Владислав Валерьевич
Баврина Алина Юрьевна

Самарский национальный исследовательский университет
имени академика С.П. Королева
443086, Самара, Московское шоссе, 34

Отпечатано на кафедре «Геоинформатика и информационная безопасность»
Тираж 20 экз.